

OpsAI

AI Business Operating System

Master Product Requirements Document — v1.0

Status: Product foundation / implementation source of truth

Purpose: Define the product, architecture, learning scope, engineering controls, and measurable outcomes for a six-month AI-native development journey.

1. Executive Summary

OpsAI is a multi-tenant AI operating layer for companies. It provides employees with a single interface to understand company knowledge, retrieve authorized business information, and eventually execute approved workflows through controlled tools and integrations. The product is intentionally designed as both a credible portfolio product and a practical GenAI engineering laboratory.

2. Product Vision

Core promise: Ask → Understand → Find → Decide → Act.

OpsAI should become an intelligent layer connecting company knowledge, structured business data, company policies, and authorized business tools while keeping business rules and security enforcement outside the LLM.

3. Target Users & Roles

Role	Primary needs
Employee	Ask about policies/processes, find information, understand work-related knowledge, access permitted business data.
Manager	Team information, activity summaries, pending work, reports, permitted operational data.
Company Admin	Users, roles, knowledge sources, integrations, permissions, approval policies, audit and AI usage.

4. Core Problem

Company information is fragmented across PDFs, documents, SOPs, internal documentation, databases, CRM/ticketing/project systems, and communication channels. Employees repeatedly search for information or depend on other employees to answer questions. This creates information fragmentation, communication gaps, delays, and inconsistent understanding.

5. Product Principles

- **RAG retrieves knowledge.** It should not be the source of truth for transactional business state.
- **DB/API provides structured business state.**
- **Backend enforces authorization and business rules.**
- **LLM interprets, reasons, communicates, and proposes tool actions.**
- **Agents orchestrate multi-step work; they do not bypass security boundaries.**
- **Retrieved content is untrusted data.**

- **High-risk actions require explicit approval.**
- **Every important AI capability must be measurable.**

6. Core User Journeys

Knowledge question: Employee asks “What is our leave policy?” → route to RAG → retrieve/rerank relevant sources → grounded answer → citation.

Structured data question: Employee asks “How many open tickets do I have?” → route to API/DB → authorization → retrieve current state → answer.

Action: Employee asks “Create a task for John” → identify tool → authorization → validate → execute or request approval → audit.

Complex workflow: User asks OpsAI to investigate a customer issue → agent orchestrates knowledge retrieval + structured data + tools + approval where needed.

7. Functional Requirements

Area	Requirements
Tenant management	Company/workspace isolation; users belong to a tenant; tenant boundary enforced in application and data access.
Authentication	Authenticated users only; session/token handling; secure identity propagation.
Authorization	RBAC plus object-level checks where required; tool-specific permissions.
Knowledge ingestion	Upload/import supported documents; parse; metadata; deterministic processing; reprocessing/versioning.
RAG	Chunking, embeddings, vector storage, dense retrieval, sparse retrieval, hybrid retrieval, RRF, reranking, compression, grou
Business data	Controlled access to structured records through backend APIs/services.
Tools	Explicit schemas, validation, authorization, retries/idempotency where applicable, safe execution.
Agents	Stateful multi-step workflows, routing, tool use, retries, approval transitions, persistence.
MCP	Introduce after conventional tools work; use where it provides meaningful interoperability.
Memory	Separate current context, workflow state, persistent memory, and knowledge base.
Evaluation	Golden datasets and automated evaluation for retrieval, generation, tools, and workflows.
Observability	Traces, logs, latency, tokens/cost, errors, retrieval and tool telemetry.

8. MVP Scope

The MVP proves the central knowledge-assistant loop without attempting full autonomy.

Multi-tenant foundation → authentication → company workspace → document upload → ingestion → chunking → embeddings → vector storage → retrieval → RAG generation → grounded answers → citations → basic employee chat.

Explicitly out of MVP: complex autonomous agents, broad external integrations, Kubernetes, advanced MCP workflows, financial actions, and unrestricted write operations.

9. Final Architecture

Frontend (Next.js/TypeScript) → FastAPI/API/Auth → OpsAI Orchestrator → request routing → RAG path / Data path / Agent path → LLM → backend validation → authorization → human approval when required → execution → audit/evaluation/observability.

10. RAG Architecture

Indexing: Source → ingestion → parsing → metadata → chunking → embeddings → vector storage.

Query: User query → transformation → dense + sparse retrieval → hybrid fusion/RRF → reranking → context compression → grounded generation → citation/source attribution.

11. Tool & Agent Safety

- LLM output is never treated as inherently trusted.
- Tools are allowlisted and narrowly scoped.
- Backend performs authentication/authorization and semantic/business validation.
- Sensitive secrets remain outside model context wherever possible.
- High-risk irreversible actions use human approval.
- Idempotency, timeouts, retries and transaction boundaries are applied where appropriate.
- Audit events record important actions and their outcomes.

12. Security & Threat Model

Threat	Required control
Direct prompt injection	Instruction hierarchy, constrained tools, backend authorization, output validation.
Indirect prompt injection	Treat retrieved documents/web content as untrusted data; never allow content to grant privileges.
Data leakage	Tenant isolation, least privilege, authorization at object level, minimal context.
Privilege escalation	Role/tool permissions enforced by backend, not by LLM.
Tool misuse	Allowlisted schemas, validation, rate limits, approval gates.
Secret exposure	Secrets manager/environment boundary; do not place credentials in prompts.
Cross-tenant access	Tenant-scoped queries and authorization at every relevant data boundary.

13. Data Model — Initial Entities

Initial conceptual entities: Tenant/Company, User, Role, Permission, Document, DocumentVersion, Chunk, Embedding, KnowledgeSource, Conversation, Message, Task, Customer, Ticket/Order (domain-dependent), Tool, ApprovalRequest, AuditEvent, EvaluationCase, EvaluationRun.

14. Technology Stack

Layer	Preferred technology
Frontend	Next.js + TypeScript
Backend	Python + FastAPI
Database	PostgreSQL
Vector search	pgvector initially; evaluate dedicated vector DB later
Cache/state	Redis
Validation	Pydantic
RAG	Core implementation first; frameworks introduced after concepts are understood
Agent workflows	LangGraph
AI interoperability	MCP after tool architecture is established
Testing	Pytest + integration/evaluation tests
Quality	Ruff + CI
Infrastructure	Docker + Linux + Caddy/reverse proxy
CI/CD	GitHub Actions
Observability	OpenTelemetry-compatible tracing/metrics/logging

15. Six-Month Delivery & Learning Roadmap

Phase	Product capability	Learning
1	Knowledge foundation	LLM/context, ingestion, chunking, embeddings, vector search
2	Advanced RAG	BM25, hybrid retrieval, RRF, reranking, query transformation, compression, citations
3	Business data + tools	Structured outputs, function calling, APIs, authorization, validation
4	Agent workflows	State, memory, planning/routing, LangGraph, retries, human approval
5	Interoperability + reliability	MCP, evaluation, observability, security, guardrails
6	Production hardening	Redis, queues, caching, rate limiting, resilience, Docker/cloud deployment

16. GenAI Learning-to-Feature Mapping

- LLM fundamentals → request understanding, generation, model routing.
- Context engineering → selecting, compressing and prioritizing context.
- Structured outputs → reliable router/tool/agent contracts.
- Tool calling → controlled business actions.
- RAG → company knowledge.
- Embeddings/vector search → semantic retrieval.
- BM25/hybrid/RRF → robust retrieval.
- Reranking/compression → relevance and context efficiency.
- Agents → multi-step work.
- LangGraph → stateful workflows and approvals.
- Memory → user/workflow continuity.
- MCP → standardized external tool/resource connectivity.
- Evaluation → measurable AI quality.
- Observability → production diagnosis.
- Security → safe AI boundaries.
- Production engineering → reliability and scale.

17. Performance Measurement Framework

OpsAI performance must be measured as a system, not by model quality alone. We will maintain a baseline evaluation set and compare every major architecture change against it.

Layer	Key metrics	Example measurement
Retrieval	Recall@K, Precision@K, MRR, nDCG	Did the correct source appear in Top-K?
Generation	Correctness, groundedness/faithfulness, citation correctness	Did the answer follow retrieved evidence?
Tools	Tool selection accuracy, argument validity, execution success	Did the agent choose and correctly call the right tool?
Agent	Task success rate, workflow completion, unnecessary steps	Did the workflow reach the intended outcome safely?
Security	Blocked injection rate, unauthorized-action rate	Were unsafe/unauthorized requests stopped?
Latency	P50/P95/P99	End-to-end and per-stage latency.
Cost	Cost/request, tokens/request	Input/output token and provider cost.
Reliability	Error rate, timeout rate, retry rate	How often does the system fail?
Product	User task completion, helpfulness, escalation rate	Can employees actually complete work faster?

18. Evaluation Process

- Create a versioned golden dataset before claiming improvements.
- Include easy, normal, ambiguous, adversarial and failure cases.
- Separate retrieval evaluation from generation evaluation.
- Test tool selection and tool arguments independently from final language quality.
- Run regression evaluation after major prompt, model, retrieval or agent changes.
- Track latency and cost alongside quality so improvements do not silently become uneconomical.
- Keep human review for high-impact or difficult-to-judge cases.

19. GitHub & Project Governance

GitHub will be the engineering source of truth. The local PRD folder will contain the product/learning documentation and versioned artifacts. The two must stay synchronized.

- **Repository root:** code, tests, CI, README, architecture references.
- **docs/product/:** master PRD and product decisions.
- **docs/architecture/:** architecture decision records and diagrams.
- **docs/learning/:** concept notes, terminology, implementation explanations.
- **docs/evaluation/:** datasets, evaluation methodology, result summaries.
- **docs/operations/:** deployment/runbooks/incident notes.
- **CHANGELOG.md:** meaningful product/architecture changes.

Recommended repository structure

```
opsai/ ■■■ docs/ ■ ■■■ product/ ■ ■ ■■■ MASTER_PRD.md ■ ■■■ architecture/ ■ ■ ■■■  
ARCHITECTURE.md ■ ■ ■■■ ADR/ ■ ■■■ learning/ ■ ■■■ evaluation/ ■ ■■■ operations/ ■■■ src/ ■■■  
tests/ ■■■ evals/ ■■■ data/ ■■■ scripts/ ■■■ README.md ■■■ CHANGELOG.md ■■■ pyproject.toml
```

20. Definition of Done

A feature is not complete merely because the coding agent generated code. It is complete when: implementation exists; tests pass; behavior is manually inspected; failure cases are tested; security boundaries are reviewed; relevant evaluation metrics are measured; documentation is updated; Git history clearly records the change; and the developer can explain what was built and why.

21. Resume Evidence Bank

Maintain a separate evidence log during development. Only verified outcomes go onto the final resume.

- Architecture capability actually implemented.
- Number and type of tools/workflows.
- Evaluation dataset size and results.
- Retrieval/generation quality metrics.
- Latency and cost measurements.
- Security tests and blocked attack cases.
- Deployment environment and reliability results.
- Meaningful product/user outcomes.

22. Acceptance Criteria for the Final Product

- An authenticated employee can ask company knowledge questions and receive grounded answers with citations.
- The system can distinguish knowledge retrieval from structured business-data requests.
- Hybrid retrieval and reranking measurably improve or maintain retrieval quality versus baseline.
- Tool actions are permission-controlled and validated by backend services.
- High-risk actions cannot execute without required approval.
- Tenant isolation is tested.
- Prompt-injection and indirect-injection test cases are included in evaluation.
- Agent workflows expose state, failures and outcomes through observability.
- Evaluation runs provide reproducible quality measurements.
- The application can be deployed from a documented production procedure.

23. Initial Success Definition

Product success: an employee can reliably use one interface to find company knowledge and authorized business information, with safe progression toward actions.

Engineering success: the architecture is modular, testable, observable, secure and measurable.

Learning success: the developer can explain and defend each major GenAI architecture decision, inspect AI-generated implementation, debug failures, and discuss trade-offs in interviews.

24. Change-Control Rule

The PRD is versioned. Product changes should be proposed as a decision, implemented in a focused GitHub change, evaluated, and then reflected in the PRD/architecture documentation. Do not silently change architecture through coding-agent prompts.